9.6 Exercises

Reinforcement

R-9.1 How long would it take to remove the $\lceil \log n \rceil$ smallest elements from a heap that contains n entries, using the removeMin operation?

R-9.2 Suppose you set the key for each position p of a binary tree T equal to its preorder rank. Under what circumstances is T a heap?

R-9.3 What does each removeMin call return within the following sequence of priority queue ADT operations: insert(5, A), insert(4, B), insert(7, F), insert(1, D), removeMin( ), insert(3, J), insert(6, L), removeMin( ), removeMin( ), insert(8, G), removeMin( ), insert(2, H), removeMin( ), removeMin( )?

R-9.4 An airport is developing a computer simulation of air-traffic control that handles events such as landings and takeoffs. Each event has a *time stamp* that denotes the time when the event will occur. The simulation program needs to efficiently perform the following two fundamental operations:

- Insert an event with a given time stamp (that is, add a future event).
- Extract the event with smallest time stamp (that is, determine the next event to process).

Which data structure should be used for the above operations? Why?

R-9.5 The min method for the UnsortedPriorityQueue class executes in $O(n)$ time, as analyzed in Table 9.2. Give a simple modification to the class so that min runs in $O(1)$ time. Explain any necessary modifications to other methods of the class.

R-9.6 Can you adapt your solution to the previous problem to make removeMin run in $O(1)$ time for the UnsortedPriorityQueue class? Explain your answer.

R-9.7 Illustrate the execution of the selection-sort algorithm on the following input sequence: (22, 15, 36, 44, 10, 3, 9, 13, 29, 25).

R-9.8 Illustrate the execution of the insertion-sort algorithm on the input sequence of the previous problem.

R-9.9 Give an example of a worst-case sequence with n elements for insertion-sort, and show that insertion-sort runs in $\Omega(n^2)$ time on such a sequence.

R-9.10 At which positions of a heap might the third smallest key be stored?

R-9.11 At which positions of a heap might the largest key be stored?

R-9.12 Consider a situation in which a user has numeric keys and wishes to have a priority queue that is *maximum-oriented*. How could a standard (min-oriented) priority queue be used for such a purpose?

R-9.13 Illustrate the execution of the in-place heap-sort algorithm on the following input sequence: (2, 5, 16, 4, 10, 23, 39, 18, 26, 15).

R-9.14 Let T be a complete binary tree such that position p stores an element with key $f(p)$, where $f(p)$ is the level number of p (see Section 8.3.2). Is tree T a heap? Why or why not?

R-9.15 Explain why the description of down-heap bubbling does not consider the case in which position p has a right child but not a left child.

R-9.16 Is there a heap H storing seven entries with distinct keys such that a preorder traversal of H yields the entries of H in increasing or decreasing order by key? How about an inorder traversal? How about a postorder traversal? If so, give an example; if not, say why.

R-9.17 Let H be a heap storing 15 entries using the array-based representation of a complete binary tree. What is the sequence of indices of the array that are visited in a preorder traversal of H? What about an inorder traversal of H? What about a postorder traversal of H?

R-9.18 Show that the sum $\sum_{i=1}^{n} \log i$, appearing in the analysis of heap-sort, is $\Omega(n \log n)$.

R-9.19 Bill claims that a preorder traversal of a heap will list its keys in nondecreasing order. Draw an example of a heap that proves him wrong.

R-9.20 Hillary claims that a postorder traversal of a heap will list its keys in nonincreasing order. Draw an example of a heap that proves her wrong.

R-9.21 Illustrate all the steps of the adaptable priority queue call remove(e) for entry e storing (16,X) in the heap of Figure 9.1.

R-9.22 Illustrate all the steps of the adaptable priority queue call replaceKey(e, 18) for entry e storing (5, A) in the heap of Figure 9.1.

R-9.23 Draw an example of a heap whose keys are all the odd numbers from 1 to 59 (with no repeats), such that the insertion of an entry with key 32 would cause up-heap bubbling to proceed all the way up to a child of the root (replacing that child's key with 32).

R-9.24 Describe a sequence of n insertions in a heap that requires $\Omega(n \log n)$ time to process.

Creativity

C-9.25 Show how to implement the stack ADT using only a priority queue and one additional integer instance variable.

C-9.26 Show how to implement the FIFO queue ADT using only a priority queue and one additional integer instance variable.

C-9.27 Professor Idle suggests the following solution to the previous problem. Whenever an entry is inserted into the queue, it is assigned a key that is equal to the current size of the queue. Does such a strategy result in FIFO semantics? Prove that it is so or provide a counterexample.

C-9.28 Reimplement the SortedPriorityQueue using a Java array. Make sure to maintain removeMin's $O(1)$ performance.

C-9.29 Give an alternative implementation of the HeapPriorityQueue's upheap method that uses recursion (and no loop).

C-9.30 Give an implementation of the HeapPriorityQueue's downheap method that uses recursion (and no loop).

C-9.31 Assume that we are using a linked representation of a complete binary tree T, and an extra reference to the last node of that tree. Show how to update the reference to the last node after operations insert or remove in $O(\log n)$ time, where n is the current number of nodes of T. Be sure to handle all possible cases, as illustrated in Figure 9.12.

C-9.32 When using a linked-tree representation for a heap, an alternative method for finding the last node during an insertion in a heap T is to store, in the last node and each leaf node of T, a reference to the leaf node immediately to its right (wrapping to the first node in the next lower level for the rightmost leaf node). Show how to maintain such references in $O(1)$ time per operation of the priority queue ADT assuming that T is implemented with a linked structure.

C-9.33 We can represent a path from the root to a given node of a binary tree by means of a binary string, where 0 means "go to the left child" and 1 means "go to the right child." For example, the path from the root to the node storing $(8,W)$ in the heap of Figure 9.12a is represented by "101." Design an $O(\log n)$-time algorithm for finding the last node of a complete binary tree with n nodes, based on the above representation. Show how this algorithm can be used in the implementation of a complete binary tree by means of a linked structure that does not keep an explicit reference to the last node instance.

C-9.34 Given a heap H and a key k, give an algorithm to compute all the entries in H having a key less than or equal to k. For example, given the heap of Figure 9.12a and query $k = 7$, the algorithm should report the entries with keys 2, 4, 5, 6, and 7 (but not necessarily in this order). Your algorithm should run in time proportional to the number of entries returned, and should *not* modify the heap.

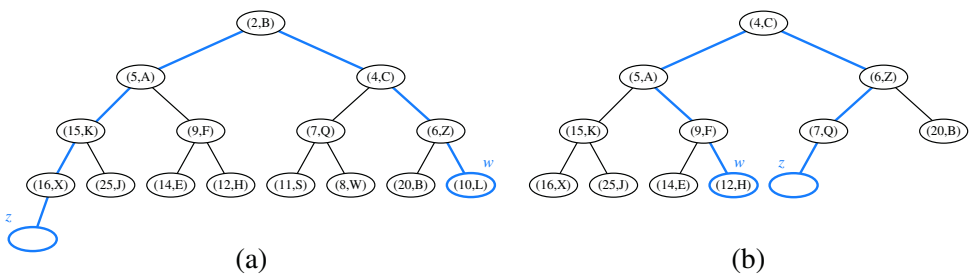

Figure 9.12: Two cases of updating the last node in a complete binary tree after operation insert or remove. Node w is the last node before operation insert or after operation remove. Node z is the last node after operation insert or before operation remove.

C-9.35 Provide a justification of the time bounds in Table 9.5.

C-9.36 Give an alternative analysis of bottom-up heap construction by showing the following summation is $O(1)$, for any positive integer h:

$$\sum_{i=1}^{h} \left(i/2^i\right).$$

C-9.37 Suppose two binary trees, T_1 and T_2, hold entries satisfying the heap-order property (but not necessarily the complete binary tree property). Describe a method for combining T_1 and T_2 into a binary tree T, whose nodes hold the union of the entries in T_1 and T_2 and also satisfy the heap-order property. Your algorithm should run in time $O(h_1 + h_2)$ where h_1 and h_2 are the respective heights of T_1 and T_2.

C-9.38 Tamarindo Airlines wants to give a first-class upgrade coupon to their top $\log n$ frequent flyers, based on the number of miles accumulated, where n is the total number of the airlines' frequent flyers. The algorithm they currently use, which runs in $O(n \log n)$ time, sorts the flyers by the number of miles flown and then scans the sorted list to pick the top $\log n$ flyers. Describe an algorithm that identifies the top $\log n$ flyers in $O(n)$ time.

C-9.39 Explain how the k largest elements from an unordered collection of size n can be found in time $O(n + k \log n)$ using a maximum-oriented heap.

C-9.40 Explain how the k largest elements from an unordered collection of size n can be found in time $O(n \log k)$ using $O(k)$ auxiliary space.

C-9.41 Write a comparator for nonnegative integers that determines order based on the number of 1's in each integer's binary expansion, so that $i < j$ if the number of 1's in the binary representation of i is less than the number of 1's in the binary representation of j.

C-9.42 Implement the binarySearch algorithm (see Section 5.1.3) using a Comparator for an array with elements of generic type E.

C-9.43 Given a class, MinPriorityQueue, that implements the minimum-oriented priority queue ADT, provide an implementation of a MaxPriorityQueue class that adapts to provide a maximum-oriented abstraction with methods insert, max, and removeMax. Your implementation should not make any assumption about the internal workings of the original MinPriorityQueue class, nor the type of keys that might be used.

C-9.44 Describe an in-place version of the selection-sort algorithm for an array that uses only $O(1)$ space for instance variables in addition to the array.

C-9.45 Assuming the input to the sorting problem is given in an array A, describe how to implement the insertion-sort algorithm using only the array A and at most six additional (base-type) variables.

C-9.46 Give an alternate description of the in-place heap-sort algorithm using the standard minimum-oriented priority queue (instead of a maximum-oriented one).

C-9.47 A group of children want to play a game, called ***Unmonopoly***, where in each turn the player with the most money must give half of his/her money to the player with the least amount of money. What data structure(s) should be used to play this game efficiently? Why?

C-9.48 An online computer system for trading stocks needs to process orders of the form "buy 100 shares at $\$x$ each" or "sell 100 shares at $\$y$ each." A buy order for $\$x$ can only be processed if there is an existing sell order with price $\$y$ such that $y \leq x$. Likewise, a sell order for $\$y$ can only be processed if there is an existing buy order with price $\$x$ such that $y \leq x$. If a buy or sell order is entered but cannot be processed, it must wait for a future order that allows it to be processed. Describe a scheme that allows buy and sell orders to be entered in $O(\log n)$ time, independent of whether or not they can be immediately processed.

C-9.49 Extend a solution to the previous problem so that users are allowed to update the prices for their buy or sell orders that have yet to be processed.

Projects

P-9.50 Implement the in-place heap-sort algorithm. Experimentally compare its running time with that of the standard heap-sort that is not in-place.

P-9.51 Use the approach of either Exercise C-9.39 or C-9.40 to reimplement the method getFavorites of the FavoritesListMTF class from Section 7.7.2. Make sure that results are generated from largest to smallest.

P-9.52 Develop a Java implementation of an adaptable priority queue that is based on an unsorted list and supports location-aware entries.

P-9.53 Write an applet or stand-alone graphical program that animates a heap. Your program should support all the priority queue operations and should visualize the swaps in the up-heap and down-heap bubblings. (Extra: Visualize bottom-up heap construction as well.)

P-9.54 Write a program that can process a sequence of stock buy and sell orders as described in Exercise C-9.48.

P-9.55 One of the main applications of priority queues is in operating systems—for ***scheduling jobs*** on a CPU. In this project you are to build a program that schedules simulated CPU jobs. Your program should run in a loop, each iteration of which corresponds to a ***time slice*** for the CPU. Each job is assigned a priority, which is an integer between -20 (highest priority) and 19 (lowest priority), inclusive. From among all jobs waiting to be processed in a time slice, the CPU must work on a job with highest priority. In this simulation, each job will also come with a ***length*** value, which is an integer between 1 and 100, inclusive, indicating the number of time slices that are needed to process this job. For simplicity, you may assume jobs cannot be interrupted—once it is scheduled on the CPU, a job runs for a number of time slices equal to its length. Your simulator must output the name of the job running on the CPU in each time slice and must process a sequence of commands, one per time slice, each of which is of the form "add job *name* with length n and priority p" or "no new job this slice".

P-9.56 Let S be a set of n points in the plane with distinct integer x- and y-coordinates. Let T be a complete binary tree storing the points from S at its external nodes, such that the points are ordered left to right by increasing x-coordinates. For each node v in T, let $S(v)$ denote the subset of S consisting of points stored in the subtree rooted at v. For the root r of T, define $top(r)$ to be the point in $S = S(r)$ with maximal y-coordinate. For every other node v, define $top(r)$ to be the point in S with highest y-coordinate in $S(v)$ that is not also the highest y-coordinate in $S(u)$, where u is the parent of v in T (if such a point exists). Such labeling turns T into a ***priority search tree***. Describe a linear-time algorithm for turning T into a priority search tree. Implement this approach.

Chapter Notes

Knuth's book on sorting and searching [61] describes the motivation and history for the selection-sort, insertion-sort, and heap-sort algorithms. The heap-sort algorithm is due to Williams [95], and the linear-time heap construction algorithm is due to Floyd [35]. Additional algorithms and analyses for heaps and heap-sort variations can be found in papers by Bentley [14], Carlsson [21], Gonnet and Munro [39], McDiarmid and Reed [69], and Schaffer and Sedgewick [82].